08 · Problem 3: The Information Bottleneck

This is the most interesting of the four problems, because it is about **quality**, not only cost.

Step 1: What happens

The only thing that passes from the LLM to the embedder is the query **text**, usually a handful of words. The embedder never sees the conversation. Whatever the LLM understood but did not write down is lost.

The paper puts it this way: the generated query is a *lossy, discrete projection* of the LLM's rich internal state.

- **Lossy**: information is thrown away.
- **Discrete**: text is made of whole words, so there is no way to express "70% this meaning, 30% that one".

Step 2: Why it hurts

A search is only as good as its query. If the query leaves out something that mattered, the right document may not come back, and the agent answers from the wrong evidence.

Step 3: Concrete examples

Conversation so far	Query the LLM writes	What got lost
Talking about **Python the programming language** for 10 turns, then: "How do I handle errors in it?"	"error handling"	*Which* language. The embedder may return Java or Rust pages.
"My **cat is 14** and has **kidney disease**. What should I feed her?"	"cat food"	Age and illness. Generic cat-food pages beat the senior renal-diet page.
"Compare the two options you just gave me."	"compare options"	*Which* options. Almost nothing useful survives.

In the toy example, the teacher and student both see "cat food". The bottleneck argument is that a student reading **hidden states** could, in principle, also carry "14-year-old, kidney disease" into the vector, because those facts shaped the hidden states even though they weren't written into the query.

Step 4: How people patched it

- **Better query rewriting:** prompt or train the LLM to write longer, self-contained queries ("senior cat kidney disease diet"). This helps, but it is still text and still a summary.
- **Bigger embedders:** a smarter reader of the same short text. It can't recover what isn't in the text.

(More in the next file.)

Step 5: What a real fix would need

Pass the LLM's **internal representation**, not just its written summary, into the search vector, while still landing in the index's space.

→ This is the motivation for the whole method (`11`, `13`).

An honest caveat

In the paper's design, the projection head reads hidden states only at the **generated query tokens**, not at the whole conversation. Those states do carry context from earlier turns (file `04`), so some of the bottleneck is plausibly avoided. However, the head is trained to **imitate the teacher**, and the teacher only sees the query text. A student trained to copy a teacher is pushed to reproduce the teacher's vector, including its blind spots. So the method removes the bottleneck *in principle*, but its training target reintroduces it *in practice*. The results (student slightly below teacher) are consistent with this. See `19`.